

Universidad San Carlos de Guatemala

Introduccion a la Computación y

Programacion

PROYECTO 2

Nombre: Pablo Gabriel Ordoñez Escobar
Carné: 202407178
Seccion: "D"
Fecha: 24/10/2025

Introducción:

Sancarlita Shop es un sistema de gestión de tienda desarrollado en Java que implementa una arquitectura MVC (Modelo-Vista-Controlador). El sistema gestiona tres tipos de usuarios: Administradores, Vendedores y Clientes, cada uno con funcionalidades específicas.

Características Principales:

- Gestión de usuarios y roles
- Catálogo de productos con categorías específicas
- Sistema de carrito de compras
- Gestión de pedidos y stock
- Reportes en PDF y CSV
- Bitácora de operaciones
- Persistencia de datos

Flujo de la Aplicación:

1. **LoginView** → Autenticación mediante **SistemaController**
2. Redirección según rol a la vista correspondiente
3. Cada módulo tiene su propio controlador
4. Los controladores interactúan con el **Sistema** (Singleton)

Patrones de Diseño Implementados

1. Singleton

- **Clase:** Sistema
- **Propósito:** Garantizar una única instancia del sistema en toda la aplicación
- **Implementación:** Constructor privado + método getInstance()

2. MVC (Modelo-Vista-Controlador)

- **Modelo:** Clases en paquete model
- **Vista:** Clases en paquete view
- **Controlador:** Clases en paquete controller

3. Herencia y Polimorfismo

- **Usuario:** Clase abstracta base para Administrador, Vendedor, Cliente
- **Producto:** Clase abstracta base para ProductoTecnologia, ProductoAlimento, ProductoGeneral

4. Factory Method (Implícito)

- Creación de productos según categoría en AdminController

Descripción de Paquetes

1. Paquete com.sancarlista.model

Clases Principales:

Sistema (Singleton)

```
public class Sistema implements Serializable {  
    // Vectores para almacenamiento  
    private Usuario[] usuarios;  
    private Producto[] productos;  
    private Pedido[] pedidos;  
    private MovimientoStock[] historialStock;  
    private RegistroBitacora[] bitacora;  
  
    // Métodos principales  
    public boolean agregarUsuario(Usuario usuario);  
    public Usuario buscarUsuarioPorCodigo(String codigo);  
    public boolean agregarProducto(Producto producto);  
    public Producto buscarProductoPorCodigo(String codigo);  
    public boolean agregarStock(String codigoProducto, int cantidad, String codigoUsuario);  
    public boolean crearPedido(Pedido pedido);  
    public boolean confirmarPedido(int idPedido, String codigoVendedor);  
    public Usuario iniciarSesion(String codigo, String contrasena);  
    public void registrarBitacora(String operacion, String estado, String descripcion);  
}
```

Jerarquía de Usuarios:

```
public abstract class Usuario {  
    protected String codigo, nombre, genero, contrasena;  
    public abstract String getTipoUsuario();  
}  
  
public class Administrador extends Usuario {  
    @Override  
    public String getTipoUsuario() { return "ADMINISTRADOR"; }  
}  
  
public class Vendedor extends Usuario {  
    private int ventasConfirmadas;  
    public void incrementarVentas() { this.ventasConfirmadas++; }  
}  
  
public class Cliente extends Usuario {  
    private String cumpleaños;  
    private String codigoVendedor;  
}
```

Jerarquía de Productos:

```
public abstract class Producto {
    protected String codigo, nombre, categoria;
    protected int stock, cantidadVendida;
    protected double precio;

    // Métodos abstractos para polimorfismo
    public abstract String getDetalleEspecifico();
    public abstract void setAtributoEspecifico(String valor);
}

public class ProductoTecnologia extends Producto {
    private int mesesGarantia;
    @Override
    public String getDetalleEspecifico() {
        return "Meses de Garantía: " + mesesGarantia;
    }
}

public class ProductoAlimento extends Producto {
    private String fechaCaducidad;
    // Implementación similar...
}

public class ProductoGeneral extends Producto {
    private String material;
    // Implementación similar...
}
```

Clases de Gestión:

- CarritoCompras: Gestiona items del carrito
- Pedido: Representa un pedido con items
- MovimientoStock: Registra cambios en inventario
- RegistroBitacora: Log de operaciones del sistema

2. Paquete com.sancarlista.view

Vistas Principales:

LoginView

- Campos: código y contraseña
- Botones: Iniciar Sesión, Salir

AdminView (JTabbedPane)

- Pestaña Vendedores: CRUD de vendedores + tabla
- Pestaña Productos: CRUD de productos + tabla + detalle

VendedorView (JTabbedPane)

- Pestaña Productos: Gestión de stock
- Pestaña Clientes: CRUD de clientes
- Pestaña Pedidos: Confirmación de pedidos

ClienteView (JTabbedPane)

- Pestaña Catálogo: Lista productos + agregar al carrito
- Pestaña Carrito: Gestión de items + realizar pedido
- Pestaña Historial: Pedidos confirmados

3. Paquete com.sancarlista.controller

Controladores:

SistemaController

- Maneja autenticación y redirección por roles
- Método iniciarSesion() y abrirModuloSegunRol()

AdminController

- Gestión completa de vendedores y productos
- Validaciones de datos y formato
- Carga de datos CSV

VendedorController

- Gestión de stock y clientes
- Confirmación de pedidos
- Exportación de historiales

ClienteController

- Gestión de carrito de compras
- Realización de pedidos
- Visualización de historial

4. Paquete com.sancarlista.util

CSVUtil

```
public class CSVUtil {  
    public static String[][] leerCSV(String rutaArchivo);  
    public static boolean escribirCSV(String rutaArchivo, String[][] datos);  
    public static boolean exportarBitacoraCSV(RegistroBitacora[] registros, String nombreArchivo);  
}
```

PDFUtil (Requiere iText 5.x)

```
public class PDFUtil {  
    public static boolean generarReporteMasVendidos(Producto[] productos);  
    public static boolean generarReporteMenosVendidos(Producto[] productos);  
    public static boolean generarReporteInventario(Producto[] productos);  
    public static boolean generarReporteVendedores(Usuario[] vendedores);  
    public static boolean generarReporteFinanciero(Producto[] productos);  
    public static boolean generarReporteClientesActivos(Usuario[] clientes, Pedido[] pedidos);  
    public static boolean generarReporteProductosPorCaducar(Producto[] productos);  
}
```

Descripción Detallada de Clases

Clase Sistema (Core del Sistema)

Responsabilidades:

- Gestión centralizada de todos los datos
- Autenticación de usuarios
- Registro en bitácora
- Serialización de datos
- Monitoreo mediante hilos

Métodos Clave:

```
// Gestión de usuarios
public boolean agregarUsuario(Usuario usuario);
public Usuario buscarUsuarioPorCodigo(String codigo);

// Gestión de productos
public boolean agregarProducto(Producto producto);
public Producto buscarProductoPorCodigo(String codigo);

// Gestión de stock
public boolean agregarStock(String codigoProducto, int cantidad, String codigoUsuario);

// Gestión de pedidos
public boolean crearPedido(Pedido pedido);
public boolean confirmarPedido(int idPedido, String codigoVendedor);

// Autenticación
public Usuario iniciarSesion(String codigo, String contrasena);

// Persistencia
public void guardarDatos();
public void cargarDatos();
```

Gestión de Vectores

El sistema utiliza arrays nativos en lugar de ArrayList:

```
private Usuario[] usuarios = new Usuario[CAPACIDAD_INICIAL];
private int cantUsuarios = 0;

// Expansión dinámica
private void expandirVectorUsuarios() {
    Usuario[] nuevoVector = new Usuario[usuarios.length * 2];
    for (int i = 0; i < cantUsuarios; i++) {
        nuevoVector[i] = usuarios[i];
    }
    usuarios = nuevoVector;
}
```

Persistencia de Datos

Serialización Java

```
public void guardarDatos() {  
    try (ObjectOutputStream oos = new ObjectOutputStream(  
        new FileOutputStream("sistema.ser"))) {  
        oos.writeObject(this);  
    } catch (IOException e) {  
        System.err.println("Error al guardar datos: " + e.getMessage());  
    }  
}
```

Archivos CSV

- Importación/Exportación de datos
- Formato personalizado sin librerías externas
- Manejo de comillas y campos especiales

Manejo de Hilos

El sistema implementa 3 hilos de monitoreo:

```
// Hilo 1: Monitor de sesiones activas (cada 10 segundos)  
hiloSesiones = new Thread(() -> {  
    while (hilosActivos) {  
        // Monitoreo de usuarios activos  
        Thread.sleep(10000);  
    }  
});  
  
// Hilo 2: Simulador de pedidos pendientes (cada 8 segundos)  
hiloPedidos = new Thread(() -> {  
    while (hilosActivos) {  
        // Monitoreo de pedidos  
        Thread.sleep(8000);  
    }  
});  
  
// Hilo 3: Generador de estadísticas (cada 15 segundos)  
hiloEstadisticas = new Thread(() -> {  
    while (hilosActivos) {  
        // Estadísticas del sistema  
        Thread.sleep(15000);  
    }  
});
```

Utilidades

CSVUtil

- **lecturaCSV:** Parsea archivos CSV considerando comillas
- **escribirCSV:** Genera archivos CSV con formato correcto
- **exportarBitacoraCSV:** Exporta registros de bitácora

PDFUtil

Genera 7 tipos de reportes:

1. Productos Más Vendidos (Top 5)
2. Productos Menos Vendidos (Top 5)
3. Inventario con Stock Crítico
4. Ventas por Vendedor
5. Clientes Activos
6. Reporte Financiero por Categoría
7. Productos por Caducar (solo alimentos)

Consideraciones Técnicas

Validaciones Implementadas

- Formato de fechas (DD/MM/YYYY)
- Stock suficiente para pedidos
- Códigos únicos de usuarios y productos
- Contraseñas con longitud mínima
- Precios y cantidades positivas

Manejo de Errores

- Try-catch en operaciones críticas
- Mensajes de error informativos
- Registro en bitácora de fallos
- Validación de datos de entrada

Seguridad

- Autenticación por código y contraseña
- Control de acceso por roles
- Validación de permisos en controladores
- Bitácora de todas las operaciones